ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/220113218

Speedup for a periodic subgraph miner

Article *in* Information Processing Letters · May 2011

DOI: 10.1016/j.ipl.2011.02.016 · Source: DBLP

CITATIONS
5

READS
45

3 authors, including:

Cinzia Pizzi
University of Padova

42 PUBLICATIONS **320** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project

COMPOSITIONAL APPROACHES FOR THE CHARACTERIZATION AND MINING OF OMICS DATA View project

All content following this page was uploaded by Cinzia Pizzi on 19 September 2017.

The user has requested enhancement of the downloaded file.

Speedup for a Periodic Subgraph Miner☆

Alberto Apostolico[a,b,1], Manuel Barbares[b], Cinzia Pizzi[b,*]

[a] *College of Computing, Georgia Institute of Technology, USA*
[b] *Dipartimento di Ingegneria dell'Informazione, Università di Padova, Italy*

Abstract

Given a series $\mathcal{G} = \{G_0, G_1, \ldots, G_T\}$ of graphs encompassing V vertices and E edges, a periodic graph is a spatially as well as temporally maximal subgraph of a subsequence of $\mathcal{G}$ in the form $\mathcal{G}_i^p = \{G_i, G_{i+p}, \ldots, G_{i+np}\}$, where n is not smaller than some predetermined threshold value σ. An algorithm for finding all such subgraphs is proposed taking time $O((E+V)(T^2 \ln(T/\sigma))$, which is faster by a factor of T than the method previously available.

Keywords: Design and analysis of algorithms, Dynamic networks, Periodic graph.

1. Introduction

Assume that a *dynamic network* [2, 5] is observed consisting of a time series of graphs $\mathcal{G} = \{G_0, G_1, \ldots, G_T\}$ drawn over a possibly evolving set of vertices. The generic element $G_t = (V_t, E_t)$ in the series represents the graph of *interactions* E_t observed at timestep t, among the set of uniquely labeled entities V_t.

☆**This is the preprint of the article published by Information Processing Letters, 111(11):521-523, 2011, available at https://doi.org/10.1016/j.ipl.2011.02.016**

*Corresponding author. Work Supported in part by the Research Program of the University of Padova.

Email addresses: axa@cc.gatech.edu (Alberto Apostolico), manuel.barbares@gmail.com (Manuel Barbares), cinzia.pizzi@dei.unipd.it (Cinzia Pizzi)

[1] Work Supported in part by the United States-Israel Binational Science Foundation (BSF) Grant No. 2008217

Let the graph $G = (V, E)$ be obtained by taking the union of all V_i's and E_i's. We encode G into two lists of integers, one for the vertices and one for the edges, as follows. Initially, the vertices of V_1 taken in some order are encoded in the first $|V_1|$ natural numbers. Consequently, each edge consists of some pair (i, j) of integers that correspond to the two adjacent vertices, and we make the convention that $i > j$. The list of edges consists of the encoding of edges in lexicographic order. If a new vertex is introduced at time t, it is encoded in the smallest natural number unused so far, and its edges are similarly encoded and appended to the edge list. It is easily seen that in this way any G_t is encoded into two ordered subsequences of what would be the encoding of the entire G, so that problems of graph containment or intersection translate into trivial linear time tasks of subsequence matching.

We are interested in the detection of graphs in the sequence $\mathcal{G}$ that recur at regular intervals. Let, for some integer p, $\mathcal{G}_i^p = \{G_i, G_{i+p}, \ldots, G_{i+np}\}$ be a subsequence of $\mathcal{G}$ such that n is not smaller than some predetermined threshold value σ. Then, any subgraph of every graph in the sequence is a *periodic* subgraph of G. Among periodic subgraphs, of particular interest are those that cannot be enriched by adding new edges without losing support. Clearly, there is only one such graph for any subsequence such as the above, and this is the largest subgraph common (MCS) to all graphs in the sequence, that is, the intersection $\tilde{G}$ of all elements of $\mathcal{G}$. If it is impossible to extend $\mathcal{G}_i^p$ by one period in either direction without changing the corresponding MCS $\tilde{G}$, then we also say that $\mathcal{G}_i^p$ is a *periodic subgraph embedding* or PSE.

Note that a closed periodic graph may be embedded into several time sequences, perhaps pertaining to distinct periods. We want to output the embeddings that correspond to each such subgraph.

By counting the number of PSE's that can be extracted from the series $\mathcal{G}$, it is shown in [3, 4] that the number of closed periodic subgraphs of minimum support σ is $O(T^2 \ln(T/\sigma))$, which subtends to the derivation of an $O((E+V)(T^3 \ln(T/\sigma))$ time algorithm assuming the population is known in advance.

In what follows, we produce an algorithm taking time $O((E+V)(T^2 \ln(T/\sigma))$. In the light of the encoding scheme, the algorithm does not need to know the population in advance. When a new member of V appears at some timestep t, it is added online to the encoding as described above.

2. Speedup

The approach of [3, 4] consists of keeping track of all PSE's into a suitable *pattern tree*, the generic node of which represents one subgraph and all descendants of a node are subgraphs of the graph at that node. Each node is suitably labeled by the list of PSE's within which the graph has been observed. Each such PSE is encoded by the triplet (i, p, s), denoting the first timestep, the period and the support of the PSE, in this order. From the triplet, we know that the last timestep in the PSE is $t_j = t_i + p(s-1)$, so that next expected timestep is $t_e = t_j + p$. The descriptor is *live* until $t < t_e$, it may or may not retain this status at time t_e but will not gain it back once it loses it. If t_i brings a graph never seen before then G_i is inserted at some compatible place in the tree and a special *anchor* descriptor is set up, so as to initialize a potential future PSE.

At the generic timestep t, G_t is read in and tested for possible updates at every node of the current tree. If the graph represented at a node is a subgraph of G_t (this requires an $O(V+E)$ time test, but in the affirmative every descendant node will not require any action), then t is added to the support, or the descriptor is removed, or nothing is done depending on which one of the conditions $t = t_e, t > t_e$ or the and $t < t_e$ is met. The algorithm finally inserts a node corresponding to G_t, in case it does not already exist, which will serve as an anchor to possibly generate a PSE with graphs seen in the future. We refer to [3, 4] for the many details here omitted for brevity. The algorithm performs one visit of the tree at every timestep t matching G_t against the graph associated with each tree node and checking every descriptor. In the worst case, the number of the nodes is equal to the number of PSE's, that are $O(T^2 \ln(T/\sigma))$, and since checking a graph at a tree node costs $(V+E)$ then the time complexity in the worst case is $O((V+E)T^3 \ln(T/\sigma))$.

In the above approach, one potential source of inefficiency is the exhaustive visit of the pattern tree performed at every timestep. An alternative is to partition the search, for each period p, among p disjoint domains of the form $\mathcal{G}_x^p = \{G_x, G_{x+p}, \ldots, G_{x+np}\}, x = 1, 2, \ldots, p-1$ so that at time t we only need to compute the MCS $\mathcal{G}_x^p$ such that $t = x \pmod p$. The creation of the lists makes it such that every PSE for period p is a substring of one of the lists. Therefore, if we compute the intersection of the graphs in every such substring this accounts for every possible PSE and corresponding periodic graph for p. Given a sequence of graphs $G_{i_1}, G_{i_2}, \ldots, G_{i_k}$ the MCS of this

sequence can be obtained using the obvious fact that

$$\mathrm{MCS}(G_{i_1}, G_{i_2}, \ldots, G_{i_k}) \equiv \mathrm{MCS}(G_{i_k}, \mathrm{MCS}(G_{i_1}, G_{i_2}, \ldots, G_{i_{k-1}})),$$

so that previously computed intersections can be used to compute the new ones. The operation of the new algorithms is as follows.

At time t, the MCS graphs present themselves partitioned in p lists for each feasible p. The generic list for p contains descriptors of PSE's encountered up this time, one descriptor per list element, in their order of appearance. The p lists start each with one of p different *phases*. The key observation is

Observation 1. *At time t, for every period p, only the list of phase $t \pmod p$ is susceptible to update.*

In order to gain instantaneous access at time t to the appropriate lists for every period at play, the algorithm uses a triangular array M such that $M[m][p]$ stores the head of the the list corresponding to m-th phase of period p.

The algorithm finds the lists corresponding to the appropriate phase of every period. For period p, it adds the anchor G_t at the head of the list, if needed, and proceeds to compute the MCS between G_t and the graph in every other element in the list. The action to be performed at the generic location of the list now depends on the MCS $C \equiv F \cap G_t$ between G_t and the graph F represented at that location.

- If $F \subseteq G_t$, i.e., $F \equiv C$, then MCS is F, and we only need to update the descriptor to reflect that t the last timestep at which F has been seen. In order to keep track of support, the algorithm also increments this count. We observe that, by construction, the graph G_{i_j} associated with a successor of the current list element is a subgraph of F. Since F is subgraph of G_t, then so is G_{i_j} ($G_{i_j} \subseteq F \subseteq G_t$). Therefore the algorithm updates all successors of the current list element without calculating the MCS, which saves computation time.

- If $C \equiv \emptyset$, then G_t and F have no subgraph in common, so that clearly no action is required except possibly publishing (when support $\geq \sigma$) the PSE represented by F with the support achieved prior to time t, and then removing this location from the list. The same reasoning applies to all successors of the current node, since they are associated

to subgraphs of F. Therefore, we can publish the corresponding PSE and remove them from the list without the need to compute the MCS with G_t.

- If $C \neq \emptyset$, then the subgraph C of F represents a MCS at timestep t, indicating a smaller graph that continues the periodic behavior of F. Since F ceased its periodicity, then its PSE must be reported in output if its support is at least σ. Next, the descriptor must be changed to reflect the change in the associated graph, and this is followed by an update of the last time step and of support. The following element in the list is then considered for further processing.

Moreover, whenever the update involves a change in the graph variable, or whenever a new element is inserted in the list, the new graph is compared with the one of the preceding element. If they are equal, the predecessor is deleted, since it would represent the same graph in a smaller periodic interval, thus lacking the condition of temporal maximality.

Correctness rests on the observation that all possible PSE's are generated by the algorithm. To assess the time complexity, observe that number of MCS's processed is

$$\sum_{p=1}^{P_{max}} \sum_{m=0}^{p} \sum_{j=0}^{\lceil (T-m)/p \rceil} j$$

where $P_{max} = T/\sigma$ is the largest period achievable. The above sum is $O(T^2 \ln P_{max})$, which combined with the fact that each MCS computation takes $O(E+V)$ time leads to an overall bound

$$O((E+V)T^2 \ln P_{max}).$$

With little extra bookkeeping, it is possible to organize the graphs in a pattern tree of the kind utilized in [3, 4], reaping the same occasional savings. Like in [3, 4], however, this conveys no gain in asymptotic time complexity.

3. Experimental tests

We report the results of an experimental comparison between ListMiner, our proposed algorithm, and PSEMiner [3, 4]. We considered both real world and artificial datasets. The real world datasets are: Enron[2], email exchange

[2]Available at http://www.cs.cmu.edu/∼enron/.

	Composition		Execution Time	
Datasets	Timesteps	Verteces	ListMiner	PSEMiner
Enron	2588	82614	21	**0.8**
Reality Mining	2940	100	**32**	720
Worst case	150	65486	**216**	587
Exp1.1 (med)	800	300	**3.4**	7.4
Exp1.2 (high)	800	50	**6.6**	29.2
Exp1.3 (low)	800	3000	3.0	**0.8**
Exp2.1 (med)	2000	300	**21.1**	200
Exp2.2 (high)	2000	50	**39.5**	712
Exp2.3 (low)	2000	3000	16.3	**7.5**

Table 1: Database composition, and execution times (in seconds) of the two algorithms, the proposed ListMiner and the state-of-the-art PSEMiner, for support threshold $\sigma = 3$ and unrestricted maximum period P_{max}.

among corporate employees, and Reality Mining[1], cellphone communication among college students. The behavior of the algorithms depends on the periodic pattern density of the dataset under analysis. To capture such a dependency, we considered datasets with diverse periodic pattern densities.

As it can be seen in Table 1 ListMiner outperform PSEMiner in almost all cases. Experiments on the worst case fall in line with the theoretical analysis for which ListMiner has better performances. For low density datasets (Enron, exp.1.3, and exp 2.3) PSEMiner is actually faster, since its complexity heavily depends on the number of nodes in the pattern tree which, in this case, is very small. In conclusion, ListMiner notably outperformed PSEMiner on both high and medium density datasets.

References

[1] N. Eagle, A. Pentland. Reality mining: sensing complex social systems. *Pers Ubiquitous Comput*, 10(4):255268, 2006.

[2] J. Han, H. Cheng, D. Xin, and X. Yan. Frequent pattern mining: current status and future directions. *Data Min Knowl Discov* 15(1):5586, 2007.

[3] M. Lahiri and T.Y. Berger-Wolf. Periodic subgraph mining in dynamic networks. *Knowledge and Information Systems*, 24(3):467-497, 2010.

[4] M. Lahiri and T.Y. Berger-Wolf. Mining Periodic Behavior in Dynamic Social Networks. *Proc. of the 8th IEEE International Conference on Data Mining (ICDM 2008)*, Pisa, Italy, pg. 373–382, 2008.

[5] J. Leskovec, J. Kleinberg, and C. Faloutsos. Graphs over time: densication laws, shrinking diameters and possible explanations. *Proc. of the 11th ACM SIGKDD Intl. Conf. on Knowl. Disc. in Data Mining*, pg. 177-187, 2005.

View publication stats